

Pourquoi une base de données graphe ?

SQL vs Neo4j — du cahier des charges à la requête

Joseph Azar

Maître de conférences, UMLP · chercheur, FEMTO-ST

MG74 — Théorie des graphes & Neo4j · CFAI Exincourt, 8 juin 2026

Le déroulé de la matinée

1. Le cahier des charges & la solution SQL

le client SochauxSense, modélisation relationnelle, on bute sur une question

2. Qu'est-ce qu'une base de données graphe ?

du relationnel au NoSQL, le modèle graphe, Neo4j, les bases de Cypher

3. SochauxSense en graphe (Neo4j)

on refait le cahier des charges en graphe et on rejoue les mêmes questions

4. Le verdict

quand le graphe gagne, quand SQL reste le bon choix

Atelier scénarisé : on joue le rôle d'un **consultant** face à un vrai besoin.

Pratique : suivez maintenant, installez plus tard

- **Suivez la démo en direct** — je projette tout (DBeaver + Neo4j Desktop). Le but est de *comprendre*, pas de taper en même temps.
- **N'installez rien maintenant.** L'installation de **Neo4j Desktop** se fait à la *pause*, *chez vous* ou en *travail autonome* — on ne perd pas la matinée en installation et soucis de compatibilité (Windows / Linux / macOS). De nombreux tutoriels existent.
- Si vous avez **déjà** Neo4j Desktop, vous pouvez suivre sur votre machine (recommandé).
- **Tous les supports seront fournis à la fin** : un **dépôt GitHub** (slides PDF, cahier des charges, fichiers `.sql` et `.md`, scripts Cypher) pour tout refaire tranquillement.

Partie 1

Le cahier des charges & la solution SQL

Le client : SochauxSense SAS

Intégrateur IoT industriel (Sochaux). Mission : un réseau de **maintenance prédictive** sur une usine d'emboutissage / montage.

- Des milliers de capteurs (*vibration, température, courant*) sur presses, convoyeurs, robots.
- Ils remontent via un **maillage de passerelles** : capteur → passerelle locale → ... → passerelle *edge-core* → réseau.
- Une passerelle tombe ⇒ **tous les capteurs en aval deviennent muets.**

« On a les équipements, le réseau, les données. On a besoin du bon **modèle** pour répondre vite aux bonnes questions. »

Deux familles de questions

Où ? localiser un capteur, une machine.

Et si... ? si une passerelle tombe, *que perd-on* et *qui intervient* ?

Ce sont les « et si » qui font mal.

Le cahier des charges (extraits)

FR1 Cataloguer chaque capteur (type, unité).

FR2 Rattacher capteur → machine → zone → bâtiment → site.

FR3 Maillage hiérarchique des passerelles (relais jusqu'à l'edge-core).

FR4 Retrouver le technicien responsable d'une zone (un tech. → plusieurs zones).

FR5 Panne d'une passerelle ⇒ **tous** les capteurs/machines qui perdent la connectivité, à *toute profondeur*.

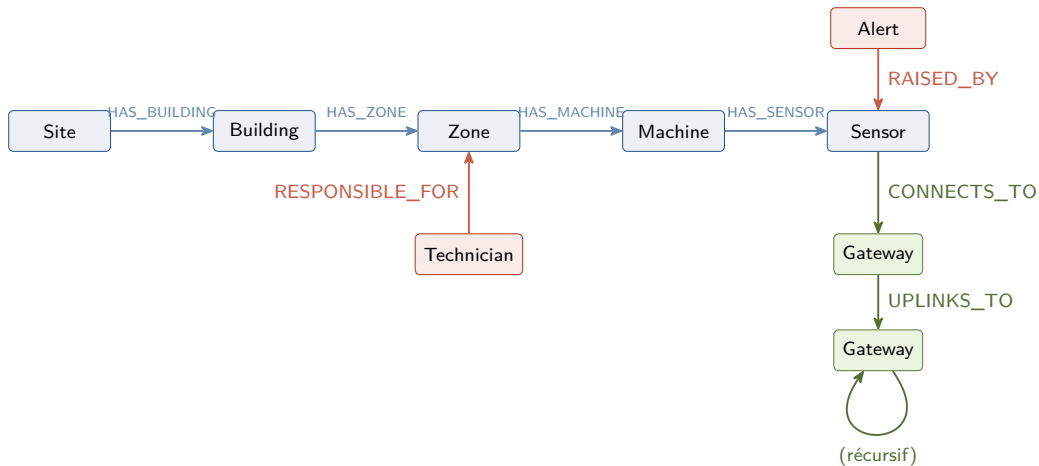
FR6 Pour un capteur, **remonter toute la chaîne** jusqu'à l'edge-core.

FR7 Identifier les machines *critiques* impactées.

FR8 Produire la **topologie complète** d'un capteur, exploitable **visuellement**.

△ FR5, FR6, FR8 : profondeur variable / orienté « chemin » — retenez-les. (on ouvre le PDF du cahier des charges)

Le modèle métier : entités & associations



Hierarchie physique · maillage réseau (profondeur variable) · organisation & événements

Un seul modèle métier — on va l'implémenter de deux façons.

Le consultant « naïf » : tout en SQL

```
1 CREATE TABLE Gateway (  
2     gateway_id      INTEGER PRIMARY KEY,  
3     parent_gateway_id INTEGER  
4     REFERENCES Gateway(gateway_id), -- auto-ref  
5     name             TEXT NOT NULL  
6 );  
7 CREATE TABLE Responsibility ( -- jonction M:N  
8     tech_id INTEGER REFERENCES Technician,  
9     zone_id INTEGER REFERENCES Zone,  
10    PRIMARY KEY (tech_id, zone_id)  
11 );
```

8 tables. Deux « pièges » à repérer :

- **Auto-référence** sur Gateway pour le maillage.
- **Table de jonction** Responsibility pour le M :N technicien/zone.

Le modèle est correct. Maintenant :

posons-lui des questions. ▶ **live DBeaver**

Le jeu de données (le même des deux côtés)

Entité	Nb	Entité	Nb
Site	1	Passerelles (3 niveaux)	5
Bâtiments	2	Capteurs	5
Zones	3	Techniciens	2
Machines	4	Alertes	3

25 nœuds au total — assez petit pour tout voir, assez profond (maillage à 3 niveaux) pour que la question clé soit intéressante.

5 questions, de la plus simple à la plus douloureuse

1. Lister les capteurs de vibration. égalité
2. Machines d'un bâtiment. + jointures
3. Technicien responsable des alertes critiques. 5 JOIN
4. Panne réseau : effet domino. le point de bascule
5. Topologie complète d'un capteur. la « forme » de la réponse

On répond à chacune en SQL maintenant ; on y reviendra en graphe.

Q1 — Capteurs de vibration

```
1 SELECT sensor_id, type, unit
2 FROM Sensor
3 WHERE type = 'vibration';
```

Une ligne utile, aucune jointure. Attendu : capteurs 1, 4, 5.

▷ [live DBeaver](#)

Q2 — Machines de l'« Atelier Emboutissage »

```
1 SELECT m.machine_id, m.model, z.name AS zone, b.name AS building
2 FROM Machine m
3 JOIN Zone      z ON m.zone_id      = z.zone_id
4 JOIN Building b ON z.building_id = b.building_id
5 WHERE b.name = 'Atelier Emboutissage';
```

2 JOIN + clauses ON. Attendu : machines 1, 2, 3.

▷ [live DBeaver](#)

Q3 — Technicien des alertes critiques

```
1 SELECT a.alert_id, a.severity, m.model AS machine, t.name AS technician
2 FROM Alert a
3 JOIN Sensor      s ON a.sensor_id = s.sensor_id
4 JOIN Machine     m ON s.machine_id = m.machine_id
5 JOIN Zone        z ON m.zone_id   = z.zone_id
6 JOIN Responsibility r ON z.zone_id = r.zone_id
7 JOIN Technician  t ON r.tech_id   = t.tech_id
8 WHERE a.severity = 'critique';
```

5 JOIN sur 6 tables. La jonction Responsibility est de la pure plomberie. Attendu : alerte 1 → Presse Schuler-2000 → Karim B.

▷ [live DBeaver](#)

Q4 — L'effet domino : il faut une *CTE* récursive

```
1  WITH RECURSIVE downstream(gateway_id, name) AS (  
2      SELECT gateway_id, name FROM Gateway WHERE gateway_id = 1  -- ancre  
3      UNION ALL  
4      SELECT g.gateway_id, g.name                                -- membre récursif  
5      FROM Gateway g  
6      JOIN downstream d ON g.parent_gateway_id = d.gateway_id  
7  )  
8  SELECT s.sensor_id, s.type, m.model AS machine  
9  FROM Sensor s  
10 JOIN downstream d ON s.gateway_id = d.gateway_id  
11 JOIN Machine  m ON s.machine_id = m.machine_id;
```

Profondeur *inconnue* → une **construction de langage** à part : ancre + membre récursif + UNION ALL. (*CTE* = requête qui s'appelle elle-même pour suivre une chaîne.)

▷ live DBeaver

Q5 — Topologie d'un capteur : une ligne plate

```
1 SELECT si.name AS site, b.name AS building, z.name AS zone,  
2         m.model AS machine, s.type AS sensor, g.name AS gateway  
3 FROM Sensor s  
4 JOIN Machine m ON s.machine_id = m.machine_id  
5 JOIN Zone z ON m.zone_id = z.zone_id  
6 JOIN Building b ON z.building_id = b.building_id  
7 JOIN Site si ON b.site_id = si.site_id  
8 JOIN Gateway g ON s.gateway_id = g.gateway_id  
9 WHERE s.sensor_id = 1;
```

Le résultat : **une seule ligne plate**. À l'humain de la recomposer en chaîne.

▷ [live DBeaver](#)

Le relationnel marche...mais Q4 a fait mal

```
1 WITH RECURSIVE chaine AS (  
2     SELECT id, parent_gateway_id, 1 AS prof  
3     FROM gateway WHERE id = :depart  
4     UNION ALL  
5     SELECT g.id, g.parent_gateway_id, c.prof+1  
6     FROM gateway g  
7     JOIN chaine c ON g.id = c.parent_gateway_id  
8 )  
9 SELECT * FROM chaine;
```

- Q1–Q3 : sans douleur (jointures, GROUP BY).
- Q4 : la relation « qui dépend de qui » **existe** (colonne parent_gateway_id)...
- ...mais c'est un **identifiant** à **re-jointre** à chaque saut, pas un lien qu'on *suit*.

Et si la relation était un objet stocké,
parcouru directement ?

Partie 2

Qu'est-ce qu'une base de données graphe ?

Pourquoi sortir du tout-relationnel ?

- Le relationnel reste **excellent** : transactions ACID, intégrité, SQL mûr (compta, stocks, RH). On ne jette rien.
- Mais 3 tensions sont apparues (web, IoT, big data) :
 - **Volume** : de très gros débits d'écriture, durs à encaisser sur un seul serveur.
 - **Schéma défini à l'avance** : changer la structure (`ALTER TABLE`) est contraignant quand les données sont hétérogènes / évolutives.
 - **Données très liées** : suivre des relations profondes = empiler les JOIN, lent et illisible.
- Réponse : la famille **NoSQL** — on choisit l'outil selon la *forme* de la donnée (et souvent on en combine plusieurs).

ACID = les garanties d'une transaction : *Atomicité* (tout ou rien), *Cohérence* (on passe d'un état valide à un autre), *Isolation* (les transactions concurrentes ne se gênent pas), *Durabilité* (une fois validé, c'est gravé — résiste aux pannes).

NoSQL n'est pas « mieux » que SQL : c'est « autre chose, pour d'autres problèmes ».

Les 4 familles NoSQL

- **Clé-valeur** (Redis) : un grand dictionnaire clé→valeur, *en mémoire* → très rapide. cache, sessions, compteurs
- **Document** (MongoDB) : un enregistrement = un document *JSON/BSON*, schéma souple. catalogues, profils
- **Colonnes larges** (Cassandra) : lignes regroupées par clé de partition, réparties sur N serveurs. gros volumes, séries temporelles
- **Graphe** (Neo4j) : les relations sont **stockées directement** entre noeuds (pas recalculées par jointure). réseaux, reco, dépendances

```
// Clé-valeur (Redis)      session:42 -> "user=alice; ttl=3600"
// Document (MongoDB)    { "_id":1, "name":"Alice", "roles":["admin","user"] }
// Colonnes larges       partition "sensor_07" : (10:00->21.4) (10:01->21.6)
// Graphe (Neo4j)         (Alice)-[:FRIEND_OF]->(Bob)-[:FRIEND_OF]->(Carol)
```

Entonnoir : quand la *relation* est la question

- « Données posées côte à côte » → clé-valeur / document / colonnes suffisent.
- « Comment X est-il **relié** à Y, en combien de sauts, par quel chemin ? » → base **GRAPHE**.
- Indices d'un cas graphe : *multi-sauts, profondeur variable, chemins, hiérarchies, réseaux*.
- Rappel Q4 : remonter une chaîne de profondeur inconnue = CTE récursive (verbeuse, perfs qui se dégradent en profondeur).

```
1 // Cypher : * = longueur variable (on borne pour eviter les cycles)
2 MATCH (a:Person {name:'Alice'})-[:FRIEND_OF*1..5]->(p)
3 RETURN DISTINCT p;
```

En graphe, parcourir des relations de proche en proche est l'opération **native** : le coût d'un saut ne dépend pas de la taille totale de la base (si l'on part d'un noeud précis).

Le modèle « property graph »

- **Noeud** : une entité (personne, machine, ville).
- **Label** : le *type* d'un noeud (:Person). Un noeud peut en porter **plusieurs**.
- **Relation** : un lien **orienté** et **typé** (:KNOWS); elle a **toujours un seul type**. Elle est *stockée à part entière*, pas recalculée par jointure.
- **Propriété** : paires clé-valeur, sur les noeuds *et* sur les relations.

Avec ces 4 briques on modélise une grande variété de domaines — mais *bien* modéliser reste un choix de conception.

```
1 (Alice:Person {age:30})  
2   -[:KNOWS {since:2020}]->  
3 (Bob:Person)
```



« Alice (Person, 30 ans) KNOWS Bob depuis 2020 ».

Graphe vs relationnel : la jointure disparaît

```
1  -- SQL : reconstruire le lien
2  SELECT b.name FROM person a
3      JOIN knows k ON k.a_id = a.id
4      JOIN person b ON b.id = k.b_id
5  WHERE a.name = 'Alice';
```

```
1  // Cypher : suivre le lien
2  MATCH (a:Person {name:'Alice'})-[:KNOWS]->(b)
3  RETURN b.name;
```

- Relationnel : la relation est une clé étrangère → reconstruite à chaque JOIN.
- Graphe : la relation est stockée et **parcourue directement**.
- *Adjacence sans index* : chaque noeud « connaît » ses voisins.
- **Le coût d'un saut ne dépend pas de la taille totale de la base** ($\neq$ JOIN).

Nuance : trouver le noeud de *départ* utilise un index ; un saut coûte $\propto$ au nombre de voisins (le degré), pas $O(1)$ absolu.

Quand le graphe brille (et quand non)

Le graphe brille

- données très **connectées** (réseaux, dépendances, topologies, hiérarchies) ;
- parcours de **profondeur variable** (descendants, plus court chemin, amis d'amis) ;
- quand les **relations** comptent autant que les données.

Choisir autre chose

- agrégations tabulaires massives (« ventes par mois ») ;
- reporting, tableaux de bord plats ;
- données simples, peu reliées, transactions plates.

Règle de poche : beaucoup de *récurtivité* / *parcours* → graphe ; beaucoup de GROUP BY → relationnel.

Un mot sur l'autre modèle : RDF

- Les deux grandes familles que vous rencontrerez :
- **Property graph** (le nôtre, Neo4j) : noeuds + relations, tous deux porteurs de propriétés.
- **RDF / triplets** (sujet-prédicat-objet) : standards W3C, requête *SPARQL*, web sémantique / données liées.
- Différence : en RDF historique, mettre une propriété *sur* un lien demande la *réification* (lourd) ; *RDF-star* (RDF 1.2) comble en partie l'écart.

```
# RDF (triplets, pour culture)
:Alice :knows :Bob .
:Alice :age    30 .
```

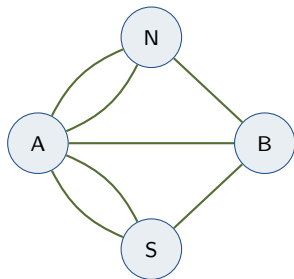
Aujourd'hui on reste sur **Neo4j = property graph + Cypher**.

Neo4j : la base de données graphe de référence

- **Neo4j** : un SGBD (système de gestion de bases de données) orienté graphe ; la base graphe la **plus populaire** (classement DB-Engines).
- **Stockage natif** : noeuds et relations stockés tels quels — pas de tables à re-joindre.
- **Transactions ACID** : même fiabilité qu'un SGBD relationnel (une transaction passe en entier, ou pas du tout).
- **Cypher** : langage *déclaratif* — on décrit *ce* qu'on veut, pas *comment* le parcourir.
- Cypher a servi de **base au standard ISO GQL** (ISO/IEC 39075 :2024) — 1^{er} langage de requête normalisé ISO depuis SQL (1987). (GQL $\neq$ GraphQL.)
- Outils : **Neo4j Desktop** (gérer ses bases) + **Neo4j Browser** (requêtes + graphe dessiné).

Aux origines : les 7 ponts de Königsberg

- Königsberg (XVIII^e s.) : une rivière, **2 rives** et **2 îles**, reliées par **7 ponts**.
- Énigme : une promenade qui emprunte *chaque pont une seule fois*?
- **1736** : Euler prouve que c'est **impossible**.
- Son idée : oublier la géographie, ne garder que les **4 morceaux de terre** (= 4 noeuds) et les **7 ponts** (= 7 arêtes).
- Acte de naissance de la **théorie des graphes** : raisonner sur des points et leurs liens.



4 terres (A,B = îles ; N,S = rives), 7 ponts

Anatomie de Cypher : un motif qui se dessine

- Un noeud entre parenthèses : `()`
- Avec son *label* (type) : `(n:Person)`
- Avec ses *propriétés* : `(n:Person {name:'Alice'})`
- Une relation entre crochets, typée : `-[:KNOWS]->`
- Orientée `(a)-[:KNOWS]->(b)` ; sans direction `(a)-[:KNOWS]-(b)`

```
1 (a:Person {name:'Alice'})  
2   -[:KNOWS]->  
3 (b:Person {name:'Bob'})
```

Le motif se **lit comme un dessin** : on tape le graphe qu'on cherche.

Ne pas confondre : *label* = le **type** du noeud (Person) ;
propriété = une **donnée** (name='Alice').

Les clauses de base : CREATE, MATCH, RETURN, WHERE

```
1 // Insérer
2 CREATE (a:Person {name:'Alice'})
3       -[:KNOWS]->
4       (b:Person {name:'Bob'});
5
6 // Lire + filtrer
7 MATCH (p:Person)-[:KNOWS]->(f:Person)
8 WHERE f.name = 'Bob'
9 RETURN p.name;
```

- CREATE $\approx$ INSERT, mais crée **toujours** (même en double); « créer si absent » = MERGE (plus tard).
- MATCH ...RETURN = le SELECT de Cypher, mais on cherche un **motif** (on peut renvoyer noeuds / relations / chemins).
- WHERE filtre comme en SQL ; une partie du filtrage est *dans le motif* (label, direction).

Teaser : -[:KNOWS*]- = **longueur variable** (1, 2, 3...sauts) — c'est ce * qui remplacera la CTE récursive du SQL.

Partie 3

SochauxSense en graphe (Neo4j)

Traduire un schéma SQL en graphe : le dictionnaire

Monde SQL	Monde graphe
Table	catégorie de noeuds = un label (:Machine)
Ligne (enregistrement)	un noeud
Colonne	une propriété ({serial:'S-042'})
Clé étrangère	une relation (:Zone)-[:HAS_MACHINE]->(:Machine)
Table de jonction M :N	une relation directe (:Technician)-[:RESPONSIBLE_FOR]->(:Zone)
Auto-référence (parent_gateway_id)	(:Gateway)-[:UPLINKS_TO]->(:Gateway)

On ne *perd* rien : la colonne / la table de jonction ne sont plus stockées comme telles, leur information **devient la relation**. (Ici on construit avec CREATE sur une base vide ; MERGE = « trouve ou crée » pour éviter les doublons.)

On reconstruit l'usine, puis on rejoue Q1 → Q5

1. On **crée** le modèle SochauxSense en Cypher (build.cypher)
2. Démo **Neo4j Browser** : on *voit* le graphe de l'usine s'afficher.
3. On rejoue **les mêmes 5 questions**, en rappelant à chaque fois la douleur SQL.

Comparaison **loyale** : mêmes questions métier des deux côtés — on ne choisit pas un terrain favorable.

▷ **démo live Neo4j**

Construire le graphe : extrait de build.cypher

```
1 // noeuds
2 CREATE (gEdge:Gateway {id:1, name:'GW-Edge-Core'})
3 CREATE (gA:Gateway {id:2, name:'GW-A'})
4 CREATE (s1:Sensor {id:1, type:'vibration', unit:'mm/s'})
5 // relations : la fleche UPLINKS_TO va de l'ENFANT vers la PARENTE
6 CREATE (gA)-[:UPLINKS_TO]->(gEdge)
7 CREATE (s1)-[:CONNECTS_TO]->(gA)
```

Le script complet (une seule instruction chaînée, 25 noeuds) se colle d'un bloc dans le Browser.

Ensuite : MATCH (n) RETURN n → l'usine entière se dessine.

▷ [démonstration live Neo4j](#)

Q1 — Capteurs de vibration

en graphe

```
1 MATCH (s:Sensor {type:'vibration'})
2 RETURN s.id, s.type, s.unit;
```

Rappel SQL : 1 ligne, 0 JOIN. Match nul — pour une recherche tabulaire simple, SQL est parfait.

▷ [démonstration live Neo4j](#)

Q2 — Machines de l'« Atelier Emboutissage »

en graphe

```
1 MATCH (b:Building {name:'Atelier Emboutissage'})
2     -[:HAS_ZONE]->(z:Zone)-[:HAS_MACHINE]->(m:Machine)
3 RETURN m.id, m.model, z.name AS zone, b.name AS batiment;
```

Rappel SQL : 2 JOIN. Un seul motif, qui se lit comme la question.

▷ [démonstration live Neo4j](#)

Q3 — Technicien des alertes critiques

en graphe

```
1 MATCH (a:Alert {severity:'critique'})-[:RAISED_BY]->(:Sensor)
2     <-[:HAS_SENSOR]-(m:Machine)<-[:HAS_MACHINE]-(z:Zone)
3     <-[:RESPONSIBLE_FOR]-(t:Technician)
4 RETURN a.id, a.severity, m.model AS machine, t.name AS technicien;
```

Rappel SQL : 5 JOIN sur 6 tables. La **table de jonction a disparu** : elle devient la relation RESPONSIBLE_FOR. Suivre la flèche *fait* le travail de la jointure.

▷ **démo live Neo4j**

Q4 — L'effet domino : un seul caractère

la question clé

```
1 MATCH (root:Gateway {name:'GW-Edge-Core'})
2     <-[:UPLINKS_TO*0..]-(:Gateway)
3     <-[:CONNECTS_TO]-(s:Sensor)<-[:HAS_SENSOR]-(m:Machine)
4 RETURN s.id, s.type, m.model;
```

Rappel SQL : une CTE récursive (WITH RECURSIVE, ≈10 lignes). La profondeur variable, c'est *. *0.. = « zéro saut ou plus » ; les 5 capteurs s'éteignent. (*1..3 bornerait la profondeur.)

▷ [démonstration live Neo4j](#)

Q4 bis — on retourne la flèche (FR6)

en graphe

```
1 MATCH chemin = (:Sensor {id:4})-[:CONNECTS_TO]->(:Gateway)
2               -[:UPLINKS_TO*0..]->(:Gateway {name:'GW-Edge-Core'})
3 RETURN chemin;
```

Remonter la chaîne d'un capteur = **inverser la flèche**. Rappel SQL : ce serait une *deuxième* CTE *réursive, de forme différente*. Attendu : capteur 4 → GW-B1 → GW-B → GW-Edge-Core.

▷ **démo live Neo4j**

Q5 — Topologie d'un capteur : la « forme » de la réponse en graphe

```
1 MATCH chemin = (si:Site)-[:HAS_BUILDING]->(:Building)-[:HAS_ZONE]->(:Zone)
2               -[:HAS_MACHINE]->(:Machine)-[:HAS_SENSOR]->(s:Sensor {id:1})
3 MATCH (s)-[:CONNECTS_TO]->(g:Gateway)
4 RETURN chemin, g;
```

Rappel SQL : une ligne plate à recomposer mentalement. On retourne un **chemin** → Neo4j dessine le **graphe**. La réponse a *la forme* de la chose. (Browser : régler Caption = name.)

▷ **démo live Neo4j**

Partie 4

Le verdict

Bonus : une propriété *sur* la relation

```
1 (:Technician)
2   -[:RESPONSIBLE_FOR {depuis:'2024'}]->
3   (:Zone)
```

- En SQL, une clé étrangère **ne peut pas** porter d'attribut sur le lien → il faut une table d'association dédiée.
- En graphe, la **relation porte ses propres propriétés**.
- C'est ça, « la relation est un objet à part entière ».

Le verdict du bon consultant

Question	SQL	Cypher	Nouvelle construction ?
Q1 lookup	1 ligne, 0 JOIN	1 ligne	non
Q2 jointures	2 JOIN	1 motif	non
Q3 alerte → tech.	5 JOIN / 6 tables	1 motif	non
Q4 effet domino	CTE récursive	1 motif (*)	OUI
Q5 topologie	ligne plate	graphe dessiné	—

Le coût n'est pas dans les requêtes simples : il est dans les parcours **multi-sauts**, à **profondeur variable** et orientés **chemin**.

Arbitrages honnêtes (ne caricaturons pas SQL)

Le graphe gagne quand...

- les données sont **très liées** ;
- on parcourt à **profondeur variable** (FR5/FR6) ;
- la réponse **est un chemin / un sous-graphe** (FR8) ;
- le modèle doit **évoluer** sans refonte.

SQL reste excellent quand...

- agrégations **tabulaires** massives ;
- reporting, comptabilité, transactions ;
- schéma stable et bien normalisé.

Le bon outil selon la *forme* des données — pas « le graphe gagne toujours ».

La suite

- **Aujourd'hui** : un noeud, un label, une relation, une propriété — et * pour la profondeur variable.
- **Scénario 2** : Cypher en profondeur — CREATE/MERGE, MATCH, WHERE, agrégations, ORDER BY.
- **Supports** : dépôt **GitHub** (PDF + Markdown) fourni à la fin de la séance.
- **Examen du 22/06** : théorie des graphes & Neo4j.

Questions ?